

Events In JS

Lec-12

Events in JS

- Events are the **things which happen** to HTML elements.
- Eg of events:
 1. **An HTML button is clicked**
 2. **A web page has finished loading**
 3. **The mouse moves over an element**
 4. **A keyboard key is pressed**
 5. **An HTML input field is changed**
- HTML DOM allows JavaScript to react to HTML events.
- Often when events happen, you may want to do something.
- When **JavaScript is used in HTML pages, JavaScript can react on events.**
- **JavaScript lets you execute code when events are detected.**
- HTML allows **event handler attributes, with JavaScript code**, to be added to HTML elements.
- In the following example, an **onclick** attribute (with event handler **displayDate()** as it's value), is added to a **<button>** element

Eg:

HTML

```
<h1 id="demo">Click the button below to generate current date and time</h1>
```

```
<button onclick="displayDate()">Click Me</button>
```

JS

```
// Event handler
const displayDate = () =>{
  let headingElement1 = document.getElementById('demo')
  headingElement1.innerText = Date()
}
```

Click the button below to generate current date and time

Click Me

Before clicking the button

Fri Aug 07 2026 20:29:56 GMT+0530 (India Standard Time)

Click Me

After clicking the button

Event Listeners

- Using event attributes like **onclick** which we saw in above example are easy to use, **but such way is not recommended**.
- It is recommended to use **addEventListener()** method to handle the events.

addEventListener()

- The **addEventListener()** method attaches an event handler to an specific element without overwriting existing event handlers.
- We can add **many event handlers to one element**, also we can add **many event handlers of same type** to one element.
- When using the **addEventListener()** method, the **JavaScript is separated from the HTML markup**, for better readability and **allows you to add event listeners even when you do not control the HTML markup**.
- **Syntax :**

```
target.addEventListener(eventName, listenerF, true/false)
```

where, **eventName** : name of the event you want to listen for, eg 'click', 'keydown', etc.

listenerFn : Call back function which runs when event fires, this function has **event object (e)** as default argument.

true: capturing phase

false/omitted : bubbling phase

- You can easily remove an event listener by using the **removeEventListener()** method.
- Same Eg which we saw above is rewritten using event listener in following way:

HTML

```
<h1 id="demo">Click the button  
below to generate current date  
and time</h1>  
  
<button>Click Me</button>
```

JS

```
let headingElement1 = document.getElementById('demo')  
  
document.querySelector('button').addEventListener("click",() =>{  
    headingElement1.innerText = Date()  
})
```

Event Propagation

- Event propagation is the **order in which an event travels through the DOM** when it happens on a **nested element**.
- Think of it like a pebble dropped in a pond — the click doesn't just happen and stop; it moves through the DOM tree in a defined path.
- There are three phases, in this order:
 1. **Capturing phase** — the event starts at the window, travels down through document, then down through every ancestor, until it reaches the actual element that was clicked.
 2. **Target phase** — the event reaches the exact element that triggered it.
 3. **Bubbling phase** — the event then travels back up, from the target element through all its ancestors, all the way to window.
- By default, **addEventListener** listens during the **bubbling** phase
- **Eg of Bubbling**

HTML

```
<div id="grandparent">
  <div id="parent">
    button id="child-btn">Click it</button>
  </div>
</div>
```

JS

```
let grandparent =
document.getElementById("grandparent");

let parent = document.getElementById("parent");

let childBtn = document.getElementById("child-btn");

grandparent.addEventListener("click", () => {
  console.log("grandparent clicked");
});

parent.addEventListener("click", () => {
  console.log("parent clicked");
});

childBtn.addEventListener("click", () => {
  console.log("childBtn clicked");
});
```

Output:

childBtn clicked

parent clicked

grandparent clicked

Event Propagation

- Eg of Capturing

HTML

```
<div id="grandparent">
  <div id="parent">
    <button id="child-btn">Click it</button>
  </div>
</div>
```

JS

```
let grandparent =
document.getElementById("grandparent");

let parent = document.getElementById("parent");

let childBtn = document.getElementById("child-btn");

grandparent.addEventListener("click", () => {
  console.log("grandparent clicked");
}, true);

parent.addEventListener("click", () => {
  console.log("parent clicked");
}, true);

childBtn.addEventListener("click", () => {
  console.log("childBtn clicked");
}, true);
```

Output:

grandparent clicked

parent clicked

childBtn clicked

Mouse Events in JS

1. **click** : Fires when an element is pressed and released with the primary mouse button (or tapped on touch devices).

HTML

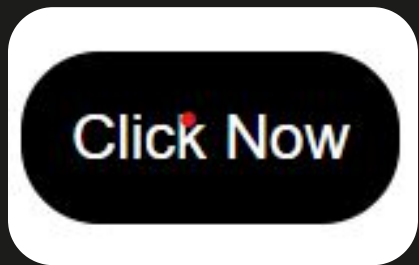
```
<button id="btn">Click Now</button>
```

JS

```
let clickNowBtn = document.getElementById("btn");

clickNowBtn.addEventListener("click", function (e) {
  clickNowBtn.innerText = "Button Clicked";
});
```

before clicking



after clicking



2. **dblclick** : Fires when an element is clicked twice in quick succession

HTML

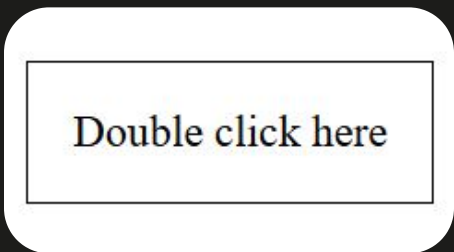
```
<div id="box">Double click here</div>
```

JS

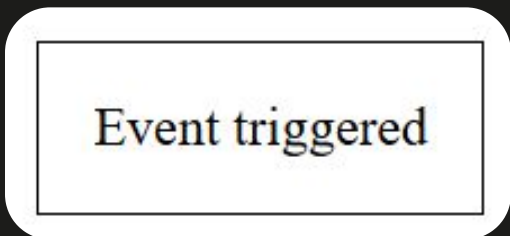
```
let box = document.getElementById("box");

box.addEventListener("dblclick", () => {
  box.innerHTML = "Event triggered";
});
```

before double clicking



after double clicking



Mouse events in JS

3. **mousedown** : Fires the moment a mouse button is pressed down over an element (before release).

HTML

```
<div id="area">Press Down Here</div>
```

JS

```
let area = document.getElementById("area");

area.addEventListener("mousedown", () => {
  area.innerHTML = "Event triggered";
});
```

4. **mouseup** : Fires when a mouse button is released over an element (after release).

HTML

```
<div id="area">Press Down Here</div>
```

JS

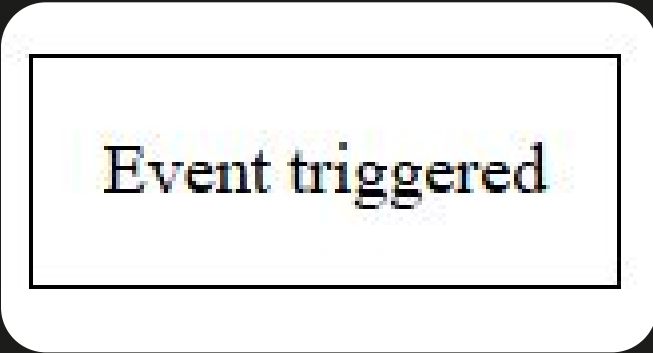
```
let area = document.getElementById("area");

area.addEventListener("mouseup", () => {
  area.innerHTML = "Back to normal";
});
```

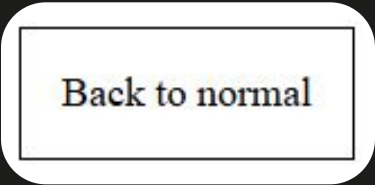
before event triggering



mousedown



mouseup



Mouse Events in JS

5. **mouseover**: Fires when the cursor moves onto an element or any of its children.

HTML

```
<div class="card-parent">
  <div class="card">Bring your mouse here</div>
</div>
```

JS

```
let card= document.querySelector('.card')
card.addEventListener('mouseover',()=>{
  document.querySelector('.card').innerText = 'Mouse
over'
})
```

6. **mouseout**: Fires when the cursor leaves an element or moves onto one of its children.

HTML

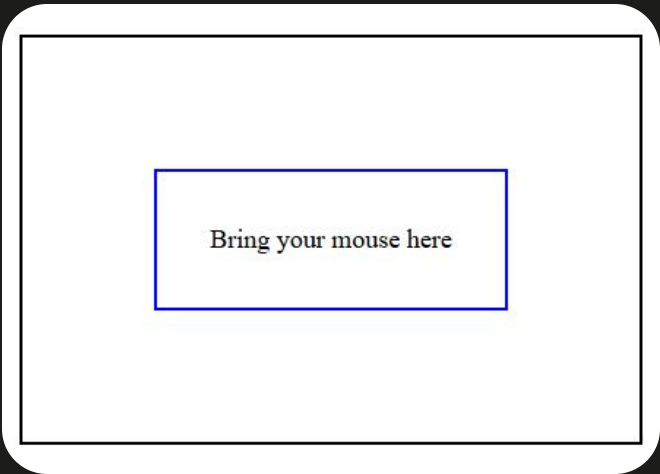
```
<div class="card-parent">
  <div class="card">Bring your mouse here</div>
</div>
```

JS

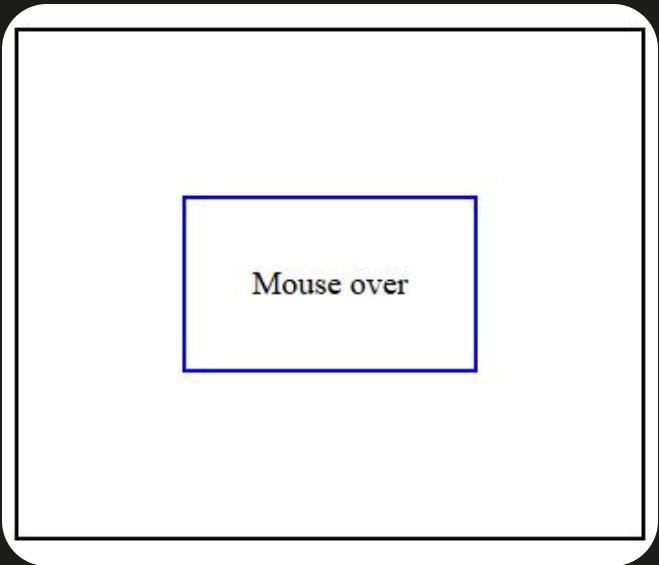
```
let card = document.querySelector('.card')

card.addEventListener('mouseout',()=>{
  document.querySelector('.card').innerText = 'Mouse
out'
})
```

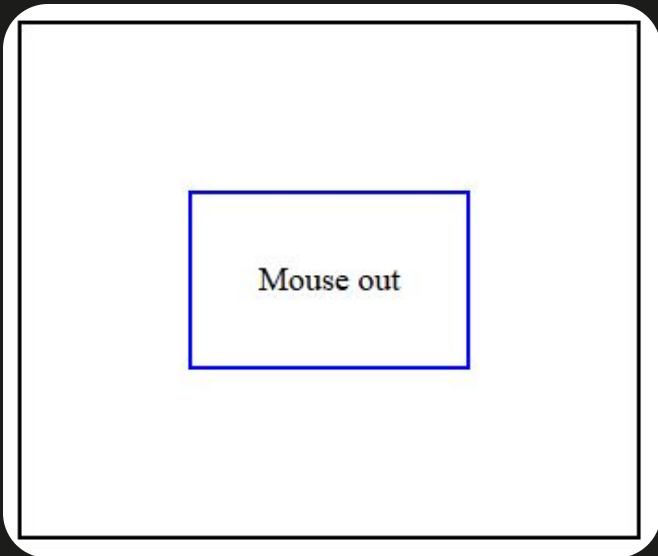
before event trigger



mouseover



mouseout



Keyboard Events in JS

1. **keydown:** Fires when a key is pressed down. Repeats continuously if the key is held.

JS

```
document.querySelector('body').addEventListener('keydown',  
(e)=>  
{  
  if(e.key === 'Enter'){  
    alert('You Pressed enter')  
  }  
})
```

Output after pressing the key

127.0.0.1:5500 says

You Pressed enter

OK

2. **keyup:** Fires once when a key is released.

JS

```
document.querySelector('body').addEventListener('keyu  
p',(e)=>  
{  
  if(e.key === 's')  
  {  
    let newElem = document.createElement('h1')  
    let newText = document.createTextNode(`You pressed  
${e.key}`)  
    let brElem = document.createElement('br')  
    newElem.appendChild(newText)  
    newElem.appendChild(brElem)  
    document.body.appendChild(newElem)  
  }  
})
```

Output after pressing key

You pressed s
You pressed s
You pressed s
You pressed s
You pressed s
You pressed s
You pressed s

📖 Self Study

- Learn more about **Form Events, Window & Document Events, Drag & Drop Events, preventDefault**

Browser Object Model (BOM)

The **Browser Object Model (BOM)** is used to **interact with the browser**.

Window Object

- The window object **represents a window in browser**.
- An window object is **created automatically by the browser** and is a **default object of the browser**.
- All global JavaScript **objects, functions, and variables with the var keyword** automatically **becomes member of the window object**.
- **Variables with let and const** keyword created globally **does not get attached to the window object**, they **live in script scope** yet they are considered global.
- **Global variables** are **properties of the window object**.
- **Global functions** are **methods of the window object**.